

Experiment 1: Linear Search

Online C Compiler

```
1 #include <stdio.h>
2
3 int main() {
4     int a[] = {10, 25, 30, 45, 50};
5     int key = 30, n = 5, i;
6
7     for(i = 0; i < n; i++) {
8         if(a[i] == key) {
9             printf("Key found at index %d", i);
10            return 0;
11        }
12    }
13
14    printf("Key not found");
15    return 0;
16 }
```

Run

Enter Input here

If your code takes input, add it in the above box before running

Output

Status : Successfully executed

Time: 0.0000 secs Memory: 1.64 Mb

Your Output

Key found at index 2

Experiment 2: Binary Search (Recursive)

Online C Compiler

```
1 #include <stdio.h>
2
3 int binarySearch(int a[], int low, int high, int key) {
4     if(low > high)
5         return -1;
6
7     int mid = (low + high) / 2;
8
9     if(a[mid] == key)
10        return mid;
11    else if(key < a[mid])
12        return binarySearch(a, low, mid - 1, key);
13    else
14        return binarySearch(a, mid + 1, high, key);
15 }
16
17 int main() {
18     int a[] = {5, 10, 15, 20, 25};
19     int key = 20;
20
21     int result = binarySearch(a, 0, 4, key);
22
23     if(result != -1)
24         printf("Key found at index %d", result);
25     else
26         printf("Key not found");
27
28     return 0;
29 }
```

Run

Enter Input here

If your code takes input, add it in the above box before running

Output

Status : Successfully executed

Time: 0.0000 secs Memory: 1.808 Mb

Your Output

Key found at index 3

Experiment 3: Factorial – Iterative and Recursive

Online C Compiler

```
C
```

```
1 #include <stdio.h>
2
3 int factorialRecursive(int n) {
4     if(n == 0)
5         return 1;
6     return n * factorialRecursive(n - 1);
7 }
8
9 int main() {
10     int n = 5, i;
11     int fact = 1;
12
13     // Iterative
14     for(i = 1; i <= n; i++)
15         fact = fact * i;
16
17     printf("Iterative Factorial = %d\n", fact);
18
19     // Recursive
20     printf("Recursive Factorial = %d", factorialRecursive(n));
21
22     return 0;
23 }
```

Run

Enter Input here

If your code takes input, add it in the above box

Output

Status : Successfully executed

Time:	Memory:
0.0000 secs	1.764 Mb

Your Output

Iterative Factorial = 120
Recursive Factorial = 120

Experiment 4: Fibonacci Series

Online C Compiler

```
C
```

```
1 #include <stdio.h>
2
3 int fibonacci(int n) {
4     if(n <= 1)
5         return n;
6
7     return fibonacci(n - 1) + fibonacci(n - 2);
8 }
9
10 int main() {
11     int n = 6, i;
12
13     printf("Fibonacci Series: ");
14
15     for(i = 0; i < n; i++)
16         printf("%d ", fibonacci(i));
17
18     return 0;
19 }
```

Run

Enter Input here

If your code takes input, add it in the above box

Output

Status : Successfully executed

Time:	Memory:
0.0000 secs	1.684 Mb

Your Output

Fibonacci Series: 0 1 1 2 3 5

Experiment 5: Matrix Multiplication

```
1 #include <stdio.h>
2
3 void merge(int a[], int low, int mid, int high) {
4     int temp[50];
5     int i = low, j = mid + 1, k = 0;
6
7     while(i <= mid && j <= high) {
8         if(a[i] < a[j])
9             temp[k++] = a[i++];
10        else
11            temp[k++] = a[j++];
12    }
13
14    while(i <= mid)
15        temp[k++] = a[i++];
16
17    while(j <= high)
18        temp[k++] = a[j++];
19
20    for(i = low, k = 0; i <= high; i++, k++)
21        a[i] = temp[k];
22 }
23
24 void mergeSort(int a[], int low, int high) {
25     if(low < high) {
26         int mid = (low + high) / 2;
27
28         mergeSort(a, low, mid);
29         mergeSort(a, mid + 1, high);
30         merge(a, low, mid, high);
31     }
32 }
33
34 int main() {
35     int a[] = {38, 27, 43, 3, 9, 82, 10};
36     int n = 7, i;
37
38     mergeSort(a, 0, n - 1);
39
40     printf("Sorted Array: ");
41
42     for(i = 0; i < n; i++)
43         printf("%d ", a[i]);
```

Enter Input here

If your code takes input, add it in the above box before

Output

Status : Successfully executed

Time:	Memory:
0.0000 secs	1.708 Mb

Your Output

Sorted Array: 3 9 10 27 38 43 82

Experiment 6: Merge Sort

Online C Compiler

C

```
1 #include <stdio.h>
2
3 int main() {
4     int A[2][2] = {{1,2},{3,4}};
5     int B[2][2] = {{5,6},{7,8}};
6     int C[2][2] = {0};
7     int i, j, k;
8
9     for(i = 0; i < 2; i++) {
10        for(j = 0; j < 2; j++) {
11            for(k = 0; k < 2; k++) {
12                C[i][j] += A[i][k] * B[k][j];
13            }
14        }
15    }
16
17    printf("Result:\n");
18
19    for(i = 0; i < 2; i++) {
20        for(j = 0; j < 2; j++)
21            printf("%d ", C[i][j]);
22        printf("\n");
23    }
24
25    return 0;
26 }
27 }
```

Run

Enter Input here

If your code takes input, add it in the above box before

Output

Status : Successfully executed

Time:	Memory:
0.0000 secs	1.652 Mb

Your Output

Result:

19 22

43 50

Experiment 7: quick Sort

```
1 #include <stdio.h>
2
3 void quickSort(int a[], int low, int high) {
4     if(low < high) {
5         int i = low, j = high;
6         int pivot = a[low];
7         int temp;
8
9         while(i < j) {
10             while(a[i] <= pivot && i < high)
11                 i++;
12
13             while(a[j] > pivot)
14                 j--;
15
16             if(i < j) {
17                 temp = a[i];
18                 a[i] = a[j];
19                 a[j] = temp;
20             }
21
22             temp = a[low];
23             a[low] = a[j];
24             a[j] = temp;
25
26             quickSort(a, low, j - 1);
27             quickSort(a, j + 1, high);
28         }
29     }
30 }
31
32 int main() {
33     int a[] = {10, 7, 8, 9, 1, 5};
34     int n = 6, i;
35
36     quickSort(a, 0, n - 1);
37
38     printf("Sorted Array: ");
39
40     for(i = 0; i < n; i++)
41         printf("%d ", a[i]);
42
43     return 0;
44 }
```

Enter Input here

If your code takes input, add it in the above box before running the code.

Output

Status : Successfully executed

Time:	Memory:
0.0000 secs	1.768 Mb

Your Output

Sorted Array: 1 5 7 8 9 10

Experiment 8: Tower of Hanoi

```
1 #include <stdio.h>
2
3 void towerOfHanoi(int n, char from, char to, char aux) {
4     if(n == 1) {
5         printf("Move disk 1 from %c to %c\n", from, to);
6         return;
7     }
8
9     towerOfHanoi(n - 1, from, aux, to);
10
11     printf("Move disk %d from %c to %c\n", n, from, to);
12
13     towerOfHanoi(n - 1, aux, to, from);
14 }
15
16 int main() {
17     int n = 3;
18
19     towerOfHanoi(n, 'A', 'C', 'B');
20
21     return 0;
22 }
```

Enter Input here

If your code takes input, add it in the above box before running the code.

Output

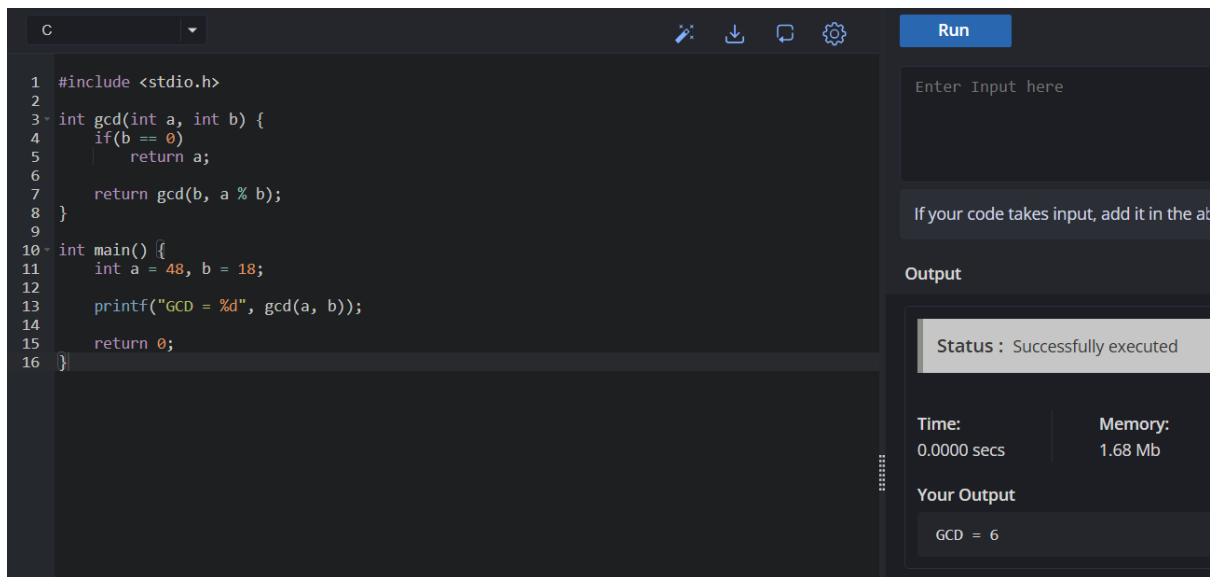
Status : Successfully executed

Time:	Memory:
0.0000 secs	1.656 Mb

Your Output

Move disk 1 from A to C
Move disk 2 from A to B
Move disk 1 from C to B
Move disk 3 from A to C
Move disk 1 from B to A
Move disk 2 from B to C
Move disk 1 from A to C

9.gcd and lcm



The screenshot shows an online C compiler interface. The code editor on the left contains a C program to calculate the Greatest Common Divisor (GCD) of two numbers, 48 and 18. The program uses a recursive function. The right sidebar shows the execution results, including a status message, execution time, memory usage, and the output.

```
1 #include <stdio.h>
2
3 int gcd(int a, int b) {
4     if(b == 0)
5         return a;
6
7     return gcd(b, a % b);
8 }
9
10 int main() {
11     int a = 48, b = 18;
12
13     printf("GCD = %d", gcd(a, b));
14
15     return 0;
16 }
```

Run

Enter Input here

If your code takes input, add it in the above box

Output

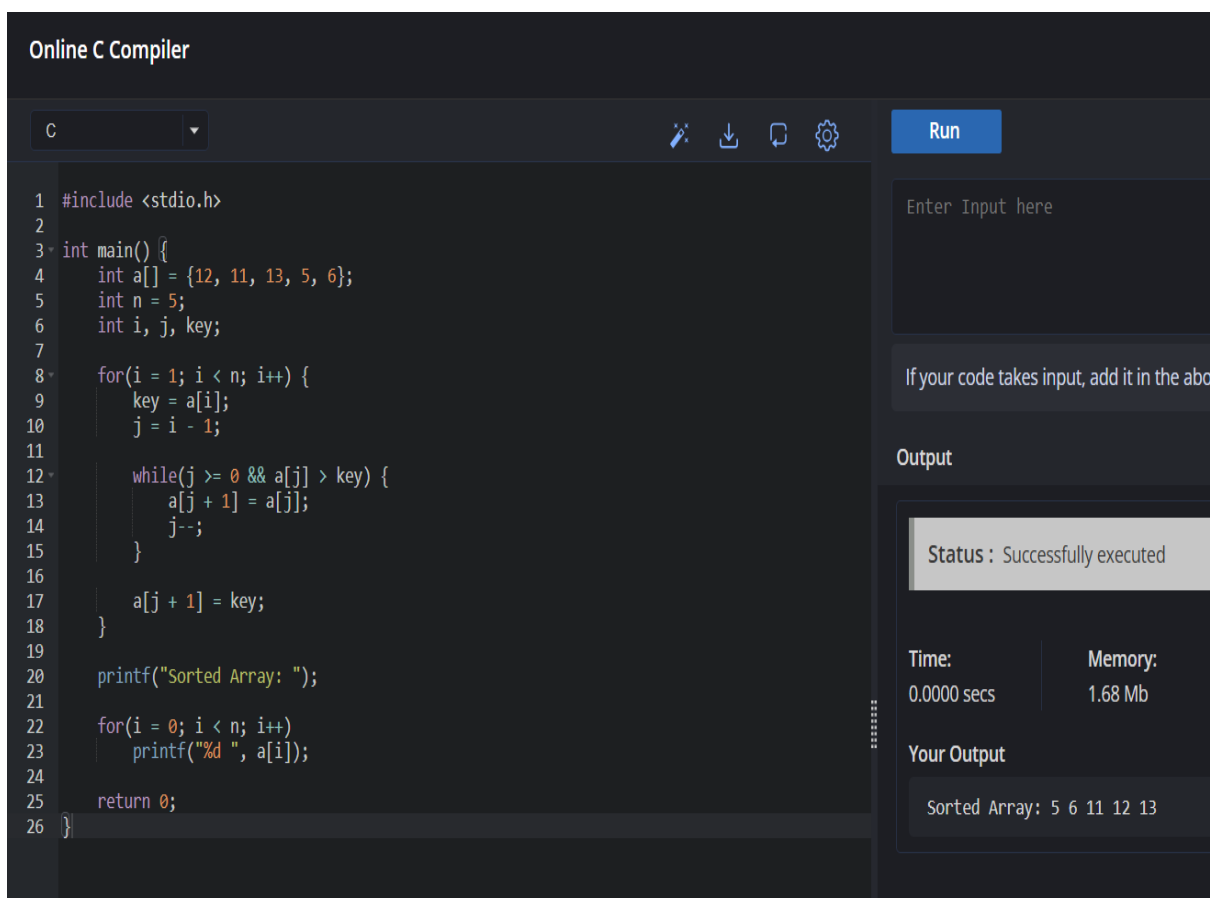
Status : Successfully executed

Time: 0.0000 secs Memory: 1.68 Mb

Your Output

GCD = 6

10.insertion sort



The screenshot shows an online C compiler interface. The code editor on the left contains a C program to sort an array of numbers using the insertion sort algorithm. The right sidebar shows the execution results, including a status message, execution time, memory usage, and the output.

```
1 #include <stdio.h>
2
3 int main() {
4     int a[] = {12, 11, 13, 5, 6};
5     int n = 5;
6     int i, j, key;
7
8     for(i = 1; i < n; i++) {
9         key = a[i];
10        j = i - 1;
11
12        while(j >= 0 && a[j] > key) {
13            a[j + 1] = a[j];
14            j--;
15        }
16
17        a[j + 1] = key;
18    }
19
20    printf("Sorted Array: ");
21
22    for(i = 0; i < n; i++)
23        printf("%d ", a[i]);
24
25    return 0;
26 }
```

Online C Compiler

Run

Enter Input here

If your code takes input, add it in the above box

Output

Status : Successfully executed

Time: 0.0000 secs Memory: 1.68 Mb

Your Output

Sorted Array: 5 6 11 12 13

11.Heap sort

```
1 #include <stdio.h>
2
3 void heapify(int a[], int n, int i) {
4     int largest = i;
5     int left = 2 * i + 1;
6     int right = 2 * i + 2;
7     int temp;
8
9     if(left < n && a[left] > a[largest])
10         largest = left;
11
12     if(right < n && a[right] > a[largest])
13         largest = right;
14
15     if(largest != i) {
16         temp = a[i];
17         a[i] = a[largest];
18         a[largest] = temp;
19
20         heapify(a, n, largest);
21     }
22 }
23
24 void heapSort(int a[], int n) {
25     int i, temp;
26
27     for(i = n / 2 - 1; i >= 0; i--)
28         heapify(a, n, i);
29
30     for(i = n - 1; i > 0; i--) {
31         temp = a[0];
32         a[0] = a[i];
33         a[i] = temp;
34
35         heapify(a, i, 0);
36     }
37 }
38
39 int main() {
40     int a[] = {4, 10, 3, 5, 1};
41     int n = 5, i;
42
43     heapSort(a, n);
44 }
```

Enter Input here

If your code takes input, add it in the above box before running.

Output

Status : Successfully executed

Time: 0.0000 secs Memory: 1.764 Mb

Your Output

Sorted Array: 1 3 4 5 10

12.counting inversion

Online C Compiler

C

```
1 #include <stdio.h>
2
3 int countInversions(int a[], int n) {
4     int count = 0;
5     int i, j;
6
7     for(i = 0; i < n; i++) {
8         for(j = i + 1; j < n; j++) {
9             if(a[i] > a[j])
10                 count++;
11         }
12     }
13
14     return count;
15 }
16
17 int main() {
18     int a[] = {2, 4, 1, 3, 5};
19     int n = 5;
20
21     printf("Number of Inversions = %d", countInversions(a, n));
22
23     return 0;
24 }
```

Run

Enter Input here

If your code takes input, add it in the above box before running.

Output

Status : Successfully executed

Time: 0.0000 secs Memory: 1.712 Mb

Your Output

Number of Inversions = 3

Experiment 13: Maximum Subarray Sum – Kadane's Algorithm

Online C Compiler

```
C
```

```
1 #include <stdio.h>
2
3 int main() {
4     int a[] = {-2, -3, 4, -1, -2, 1, 5, -3};
5     int n = 8;
6     int maxSum = a[0];
7     int currentSum = a[0];
8     int i;
9
10    for(i = 1; i < n; i++) {
11        if(currentSum + a[i] > a[i])
12            currentSum = currentSum + a[i];
13        else
14            currentSum = a[i];
15
16        if(currentSum > maxSum)
17            maxSum = currentSum;
18    }
19
20    printf("Maximum Subarray Sum = %d", maxSum);
21
22    return 0;
23 }
```

Run

Enter Input here

If your code takes input, add it in the above box

Output

Status : Successfully executed

Time: 0.0000 secs Memory: 1.648 Mb

Your Output

Maximum Subarray Sum = 7

14: Exponentiation – Fast Power

Online C Compiler

```
C
```

```
1 #include <stdio.h>
2
3 int power(int x, int n) {
4     if(n == 0)
5         return 1;
6
7     if(n % 2 == 0) {
8         int half = power(x, n / 2);
9         return half * half;
10    }
11
12    return x * power(x, n - 1);
13 }
14
15 int main() {
16     int x = 2, n = 10;
17
18     printf("%d^%d = %d", x, n, power(x, n));
19
20     return 0;
21 }
```

Run

Enter Input here

If your code takes input, add it in the above box

Output

Status : Successfully executed

Time: 0.0000 secs Memory: 1.644 Mb

Your Output

2^10 = 1024

15.closest pair of elements

```
1 #include <stdio.h>
2 #include <math.h>
3
4 int main() {
5     int points[4][2] = {
6         {1, 2},
7         {4, 5},
8         {7, 8},
9         {3, 1}
10    };
11
12    double minDistance = 999999;
13    int p1, p2;
14    int i, j;
15
16    for(i = 0; i < 4; i++) {
17        for(j = i + 1; j < 4; j++) {
18            double dx = points[i][0] - points[j][0];
19            double dy = points[i][1] - points[j][1];
20
21            double distance = sqrt(dx * dx + dy * dy);
22
23            if(distance < minDistance) {
24                minDistance = distance;
25                p1 = i;
26                p2 = j;
27            }
28        }
29    }
30
31    printf("Closest Pair: (%d,%d) and (%d,%d)\n",
32          points[p1][0], points[p1][1],
33          points[p2][0], points[p2][1]);
34
35    printf("Distance = %.2f", minDistance);
36
37    return 0;
38 }
```

Enter Input here

If your code takes input, add it in the above box

Output

Status : Successfully executed

Time:

0.0000 secs

Memory:

2.256 Mb

Your Output

Closest Pair: (1,2) and (3,1)
Distance = 2.24